Exp No: 24

MODELING CLOCK-DRIVEN CONCURRENT PROCESSES USING

AIM:

To implement a clock-driven concurrent process using sc_clock, SC_THREAD, wait(), and positive-edge sensitivity in SystemC.

PROCEDURE:

1. Create a SystemC program using SC_MODULE and declare the clock input using sc_in<bool> clk.
2. Define a process using SC_THREAD(process) and make it sensitive to the positive edge of the clock using sensitive << clk.pos().
3. Use wait() inside the process to suspend execution until the next clock event.
4. Create a clock with a period of 1 ns and connect it to the module.
5. Compile and execute the program, run the simulation for 5 ns, and observe the clock-triggered output.

PROGRAM:

```
#include <systemc.h>


SC_MODULE(ClockExample) {

sc_in<bool> clk;


void process() {

while (true) {

wait();

std::cout << "Time: " << sc_time_stamp()

<< " - Clock Triggered" << std::endl;

}

}


SC_CTOR(ClockExample) {

SC_THREAD(process);
```

```
sensitive << clk.pos();

}
};

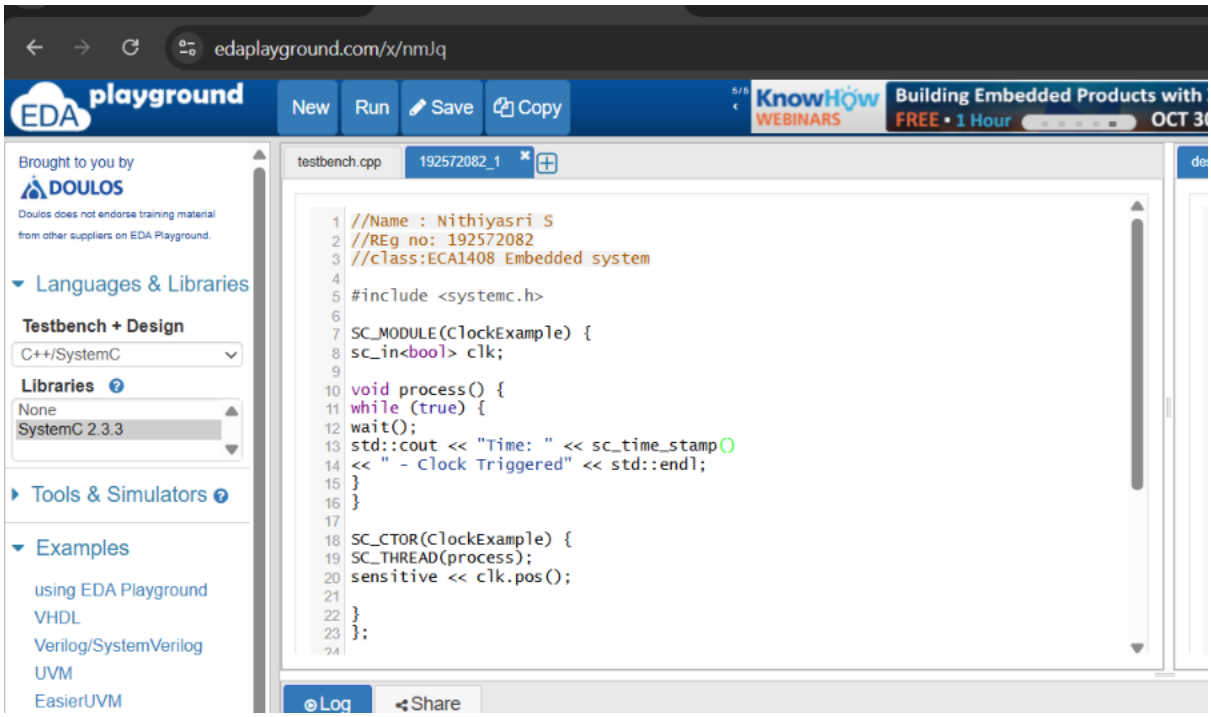
int sc_main(int argc, char* argv[]) {

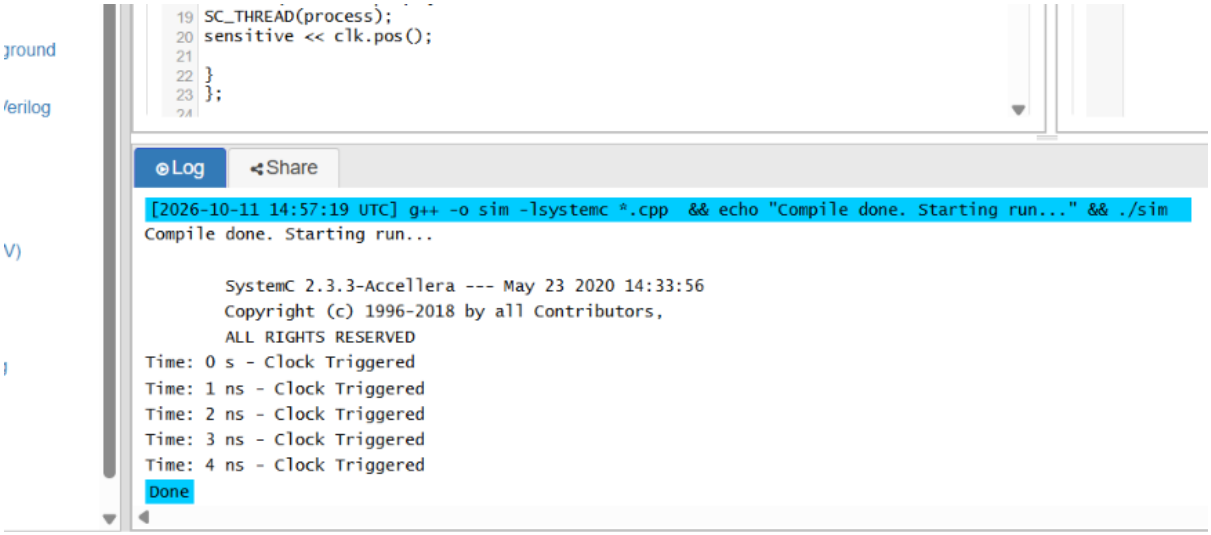
sc_clock clock("clock", 1, SC_NS);

ClockExample module("module");
module.clk(clock);

sc_start(5, SC_NS);

return 0;
}
```





RESULT:

Thus, the clock-driven concurrent process using SC_THREAD and wait() was successfully implemented and verified in SystemC.